

Clase 221 — Fundamentos de seguridad en la nube y responsabilidad compartida

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: *AWS Well-Architected Framework: Security Pillar (documentación oficial de AWS)* 🕒 Duración estimada: **90 min** · Nivel: **Fundamentos**

Objetivo

Comprender qué cambia cuando la infraestructura se mueve a la nube y dónde queda exactamente la frontera de responsabilidad entre el proveedor y el cliente. Al terminar, el alumno sabrá clasificar cualquier servicio (IaaS, PaaS, SaaS) según quién asegura qué, e identificar los errores de configuración del cliente que causan la mayoría de las brechas cloud.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el modelo de responsabilidad compartida y cómo se desplaza según el tipo de servicio.
2. **Distinguir** "seguridad DE la nube" (proveedor) de "seguridad EN la nube" (cliente).
3. **Identificar** las categorías de misconfiguration más frecuentes y su impacto.
4. **Aplicar** los cinco pilares del pensamiento Well-Architected al diseño seguro.
5. **Mapear** controles de seguridad tradicionales a sus equivalentes cloud.

Temas

#	Tema	Por qué importa
1	Modelo de responsabilidad compartida	Define legalmente y técnicamente qué te toca asegurar
2	IaaS vs PaaS vs SaaS	La frontera de responsabilidad se mueve con cada modelo
3	"DE la nube" vs "EN la nube"	Separa fallos del proveedor de errores del cliente
4	Misconfiguration como causa raíz	El 99% de brechas cloud son error del cliente (Gartner)
5	Perímetro definido por identidad	En la nube, IAM sustituye al firewall perimetral
6	Regiones, zonas y soberanía de datos	Cumplimiento, latencia y aislamiento de fallos
7	Modelo de amenazas cloud	Superficie de API, plano de gestión y datos

Definiciones y características

- **Responsabilidad compartida:** modelo donde el proveedor asegura la infraestructura subyacente y el cliente asegura lo que despliega encima. *Característica clave:* la línea divisoria depende del servicio contratado.

- **IaaS (Infrastructure as a Service):** el proveedor da cómputo, red y almacenamiento virtualizados (p. ej. EC2). *Clave:* el cliente asegura SO, parches, red y aplicación.
- **PaaS (Platform as a Service):** el proveedor gestiona también el runtime y el SO (p. ej. App Engine, RDS). *Clave:* el cliente asegura datos, configuración y accesos.
- **SaaS (Software as a Service):** el proveedor gestiona casi todo (p. ej. Microsoft 365). *Clave:* el cliente solo asegura datos, identidades y configuración de la app.
- **Plano de control (control plane):** las APIs de gestión del proveedor. *Clave:* comprometerlo da control total; se protege con IAM y MFA.
- **Misconfiguration:** ajuste inseguro dejado por el cliente (bucket público, puerto abierto). *Clave:* invisible para el proveedor, es el vector dominante.
- **Blast radius:** alcance del daño si un recurso se compromete. *Clave:* se limita con segmentación de cuentas, VPCs y privilegio mínimo.

Herramientas y preparación

- Cuenta gratuita en al menos un proveedor (AWS Free Tier, Azure free account o GCP free tier). **No subas datos reales;** usa una cuenta de laboratorio dedicada.
- CLIs oficiales instaladas: `aws`, `az`, `gcloud`.
- Habilita **MFA en la cuenta raíz/administrador** antes de cualquier práctica.

```
# Verificar instalación de CLIs
aws --version
az version
gcloud version
```

Laboratorio guiado (ejercicio aplicado)

Este es un tema conceptual: el laboratorio es un análisis de arquitectura y una matriz de responsabilidad.

1. Elige tres servicios reales, uno por modelo: por ejemplo **EC2** (IaaS), **AWS Lambda** (PaaS/FaaS) y **Amazon S3** (almacenamiento gestionado).
2. Para cada uno, construye una tabla con las capas: *hardware físico, red, hipervisor, SO, runtime, aplicación, datos, IAM/config*. Marca en cada capa quién es responsable: **Proveedor, Cliente o Compartida**.
3. Investiga en la documentación oficial cómo describe el proveedor la frontera de cada servicio y contrasta con tu tabla.
4. Toma una brecha pública real (p. ej. una fuga por bucket S3 mal configurado) e identifica **en qué capa** falló y **de quién** era la responsabilidad. Comprobarás que casi siempre fue del cliente.
5. Redacta un párrafo de "controles mínimos del cliente" para cada uno de los tres servicios.
6. Repite el ejercicio de la capa IAM: describe por qué en la nube el perímetro es la identidad y no la red.

Ejercicios

1. Dibuja el diagrama de responsabilidad compartida para IaaS, PaaS y SaaS lado a lado.
2. Clasifica 10 servicios reales de tu proveedor favorito según el modelo que representan.
3. Enumera cinco categorías de misconfiguration y asocia a cada una un control preventivo.
4. Explica con un ejemplo por qué "el proveedor es seguro" no implica "mi cuenta es segura".

5. Diseña un esquema de múltiples cuentas para separar producción de desarrollo y justifica el blast radius reducido.
6. Redacta una política interna de una página sobre uso de la cuenta raíz.

Reto verificable

Elabora una **matriz de responsabilidad compartida** en formato tabla para una arquitectura de tres capas (balanceador gestionado + contenedores + base de datos gestionada) en el proveedor que elijas.

Criterio de aceptación: cada componente lista al menos seis capas, cada capa tiene asignado responsable (Proveedor/Cliente/Compartida) coherente con la documentación oficial, y se incluye al menos un control concreto del cliente por componente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"El proveedor ya me protege, no necesito hacer nada"	Malentendiendo el modelo; la config y los datos siempre son del cliente. Aplica controles propios.
Bucket o blob accesible públicamente sin querer	ACL/política heredada o pública por defecto en versiones antiguas; activa <i>block public access</i> .
Uso diario de la cuenta raíz	Rompe el privilegio mínimo; crea usuarios/roles y guarda la raíz solo para tareas críticas con MFA.
Datos en región equivocada	Se ignoró soberanía/latencia; define región por diseño y bloquea otras con SCP/políticas.
"No sé qué recursos tengo"	Falta de inventario; habilita Config/Asset Inventory desde el día uno.

Preguntas frecuentes

? ¿La responsabilidad compartida es un contrato legal o solo técnico? Ambas cosas. Está reflejada en el acuerdo del proveedor y define técnicamente qué controles quedan de tu lado; en una brecha por misconfiguration la responsabilidad legal suele ser del cliente.

? ¿En SaaS ya no tengo nada que asegurar? Sí lo tienes: identidades, permisos, configuración de la app y los datos que subes. La mayoría de incidentes SaaS son por cuentas mal protegidas o permisos excesivos.

? ¿Por qué se dice que "la identidad es el nuevo perímetro"? Porque no hay un firewall físico entre tú y el mundo: cualquiera con credenciales válidas puede llamar a la API. Controlar quién puede hacer qué (IAM) es el control central.

Referencias

- AWS Shared Responsibility Model. <https://aws.amazon.com/compliance/shared-responsibility-model/>
- AWS Well-Architected — Security Pillar. <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html>

- Microsoft — Shared responsibility in the cloud.
<https://learn.microsoft.com/azure/security/fundamentals/shared-responsibility>
- Google Cloud — Shared responsibility and shared fate.
<https://cloud.google.com/architecture/framework/security/shared-responsibility-shared-fate>
- NIST SP 800-145, *The NIST Definition of Cloud Computing*. <https://csrc.nist.gov/pubs/sp/800/145/final>

Siguiente clase

Clase 222 - IAM en la nube: identidades, roles y permisos